Le Mans Université

Licence Informatique *2ème année*

Module 174UP02 Rapport de Projet

JurassicWar

Solène Orieux, Romane Saint-Léger, Hannah Sergent

7 avril 2026

Table des matières

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Énoncé du problème . . . 3
1.3 Objectifs initiaux du projet . . . 3

2 Conception **3**
2.1 Principales règles du jeu . . . 3
2.2 Foncionnalités prévues . . . 3

3 Organisation du travail **3**
3.1 Répartition des tâches . . . 3
3.2 Planning prévisionnel . . . 3
3.3 Outils de programmation utilisés . . . 3

4 Developpement **3**
4.1 Représentation du terrain par une matrice . . . 3
4.2 Positionnement et mobilité des dinosaures . . . 3
4.3 Système d'actions et outils à disposition du joueur . . . 4
4.3.1 Mécanique des changements de tours . . . 4
4.3.2 Infliger un coup mortel : bombe . . . 4
4.3.3 Blesser un dinosaure : armes de tir . . . 7
4.3.4 Autre méthode de déplacement : grapin . . . 7
4.4 Interface graphique et utilisateur . . . 8
4.4.1 Utilisation du module SDL . . . 8
4.4.2 Création d'un menu principal . . . 8
4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur . . . 9

5 Résultats et Conclusion **9**

6 Bibliographie **9**

7 Annexes **10**
7.1 Jeux de tests . . . 10
7.2 Traces de débogage . . . 10
7.2.1 Correction du système de collisions de la bombe . . . 10

1 Introduction

1.1 Contexte

1.2 Énoncé du problème

1.3 Objectifs initiaux du projet

2 Conception

2.1 Principales règles du jeu

2.2 Foncionnalités prévues

3 Organisation du travail

3.1 Répartition des tâches

3.2 Planning prévisionnel

3.3 Outils de programmation utilisés

4 Developpement

4.1 Représentation du terrain par une matrice

4.2 Positionnement et mobilité des dinosaures

4.3 Système d'actions et outils à disposition du joueur

Cette partie rend compte de la manière dont s'enchaînent les actions dans JurassicWar à travers la mécanique des changements de tours mais aussi de la nature des actions possibles dans le jeu. Un dinosaure peut en effet user de différents outils tels que la bombe, les armes de tirs ou encore le grapin.

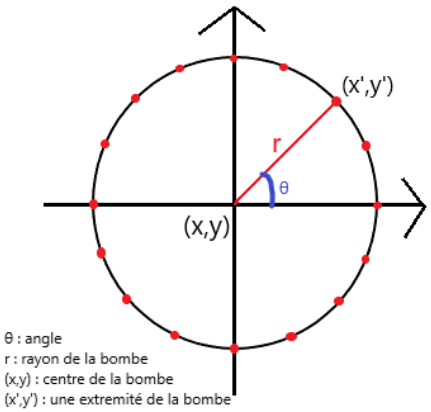
4.3.1 Mécanique des changements de tours

Premièrement, les dinosaures jouent chacun leur tour, c'est-à-dire que l'utilisateur n'a pas la possibilité de choisir avec lequel de ses trois dinosaures il souhaite agir à un tour donné. L'instauration de cette règle augmente d'une part la complexité du jeu, mais d'autre part le système des changements de tours, puisque l'alternance entre tous les dinosaures doit être correcte même lorsqu'un ou plusieurs d'entre eux meurent. Afin de mettre en œuvre les changements de tours, une fonction *tourSuivant* a été crée. Elle s'appuie sur l'identifiant associé à chaque dinosaure pour déterminer le dinosaure qui peut jouer. Cependant, comme les dinosaures de la première équipe sont numérotés de un à trois et ceux de la seconde équipe de quatre à six, à la suite du premier dinosaure, c'est au quatrième dinosaure de jouer et non pas au second dinosaure pour garantir l'alternance entre les deux équipes. Ceci nécessite de mémoriser deux identifiants : celui du dinosaure qui vient d'agir et celui du dinosaure ayant agi précédemment. Ces identifiants sont stockés dans uns structure *t_ tour* qui contient également le numéro de l'équipe qui vient de jouer et un compteur du nombre de tours effectués. Par ailleurs, la fonction *tourSuivant* doit tenir compte de l'état d'un dinosaure dans la mesure où un dinosaure mort ne peut plus jouer. Ainsi, une fois que la fonction a choisi un dinosaure, elle vérifie qu'il est vivant. Dans le cas contraire, elle sélectionne un autre dinosaure jusqu'à en trouver un encore dans la partie. Une sécurité a été ajoutée pour que la fonction ne renvoie aucun dinosaure en fin de partie, c'est à dire lorsque tous les dinosaures d'un joueur ont péri.

4.3.2 Infliger un coup mortel : bombe

Deuxièmement, au cours d'une partie, les joueurs peuvent faire appel à une arme mortelle : la bombe. Contrairement aux armes de tirs évoquées dans la section suivante, la bombe tue instantanément le dinosaure qu'elle heurte. Néanmoins, cette arme peut uniquement être employée si le dinosaure dispose d'une place suffisante à côté de lui pour la positionner. L'objectif consiste à apporter de la difficulté supplémentaire mais aussi un intérêt à utiliser chaque arme disponible dans le jeu. De surcroît, cette arme se caractérise par sa capacité à rebondir sur le terrain. Cette capacité de la bombe à rebondir induit l'utilisation de plusieurs notions mathématiques dans le code. Pour faciliter l'application de ces notions, la bombe est représentée par une structure *t_ bombe* comportant notamment le rayon et le vecteur vitesse du

projectile, c'est à dire la vitesse sur l'axe horizontal et vertical de la bombe. Tout d'abord, la mise en place des rebonds nécessite un système de collision avec le sol précis en raison du relief particulier du terrain. Le terrain étant formé de collines et de cratères aux courbures arrondies, une détection de collisions limitée aux quatre directions haut, gauche, bas et droite ne suffit pas et manque les impacts en diagonale. En conséquence, pour élargir le nombre de directions étudiées lors des collisions, la fonction *collisionTerrainBombe* fait recourt aux formules des coordonnées polaires pour analyser une multitude de points sur le contour de la bombe. Les points analysés sont représentés en rouge sur la figure 1 accompagnés de la formule pour calculer leurs coordonnées.



$$x' = x + r\cos(\theta)$$

$$y' = y + r\sin(\theta)$$

FIGURE 1 – Schéma des coordonnées polaires appliqué à la bombe

En cas de collision avérée entre un ou plusieurs de ces points et la terre, une seconde étape consiste à étudier la pente du terrain de manière à ce que la bombe rebondisse perpendiculairement au sol. Pour cela, les valeurs dans la matrice des points voisins au lieu d'impact sont examinées et les pixels contenant de la terre sont intégrés dans le calcul d'un gradient à la surface du terrain, c'est à dire d'un vecteur qui pointe vers la direction où la densité de la terre est la plus forte. Un exemple de gradient pouvant être établi par la fonction *collisionTerrainBombe* est représenté sur la figure 2. Dans le calcul du gradient, il est donné un poids important aux points jouxtant l'impact et un poids plus faible aux voisins plus éloignés afin de rendre compte au mieux du relief et pour qu'aucune éventuelle irrégularité du terrain ne fausse les résultats. De plus, dans le cas où toute une partie de la bombe est en contact avec le sol, le gradient est déterminé pour chaque point de collision. En prenant l'opposé de chacun de ces vecteurs, une famille de vecteurs normaux est formée, c'est à dire un ensemble de vecteurs perpendiculaires au terrain. Le vecteur normal finalement choisi pour le rebond constitue une moyenne de tous ces vecteurs. La bombe suit la direction de ce vecteur pour s'éloigner du terrain comme le recapitule la figure 2. Par ailleurs, le choc n'impacte pas

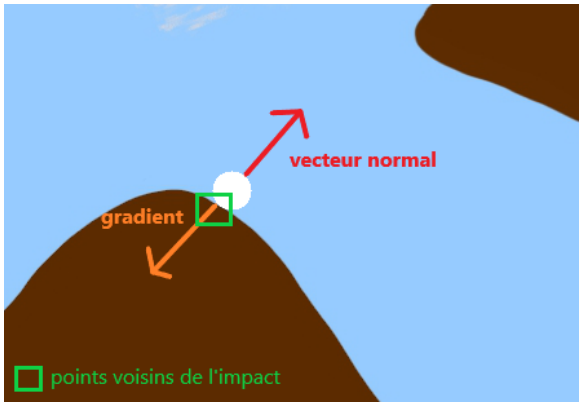


FIGURE 2 – Exemple d'un gradient et d'un vecteur normal associé à la bombe en un point donné du terrain

uniquement la trajectoire de la bombe mais également sa vitesse. La nouvelle vitesse de la bombe suite au rebond est obtenue par la formule de réflexion par rapport à un plan :

$$\vec{v}' = \vec{v} - 2\,(\vec{v} \cdot \vec{n})\,\vec{n} \tag{1}$$

où $\vec{v}$ désigne le vecteur vitesse de la bombe, $\vec{v}'$ le vecteur vitesse après rebond et $\vec{n}$ le vecteur normal. Pour procurer plus de réalisme au rebond et décrire le ralentissement se produisant dans le monde réel lorsqu'un projectile percute un objet, un coefficient de restitution intervient dans la formule 1 pour moduler la valeur du facteur deux. Ce coefficient varie entre zéro et un, où un signifie un rebond parfait et zéro une absence totale de rebond. Enfin, concernant la vitesse et la trajectoire de la bombe lorsqu'elle ne rencontre pas d'obstacle sur son chemin, celles-ci reposent sur une approximation des équations de mouvement suivantes :

$$\begin{aligned} x(t) &= x_0 + v_{x0}\,t \\ y(t) &= y_0 + v_{y0}\,t + \frac{1}{2}gt^2 \end{aligned} \tag{2}$$

où (x_0, y_0) désignent les coordonnées à l'instant présent de la bombe, (x, y) les nouvelles coordonnées calculées, $\vec{v} = (v_{x0}, v_{y0})$ le vecteur vitesse, t le temps écoulé et g la gravité. De la même manière que précédemment, l'introduction du temps et de la gravité dans l'équation a pour visée de reproduire plus fidèlement possible le comportement d'une bombe dans le monde réel.

4.3.3 Blesser un dinosaure : armes de tir

4.3.4 Autre méthode de déplacement : grapin

4.4 Interface graphique et utilisateur

La visualisation à l'écran tous les éléments créés dans le cadre du projet tels que la matrice, le positionnement des dinosaures ou encore les outils, nécessite une interface graphique. Cette interface, développée grâce au module SDL, constitue la base de l'interface utilisateur du jeu en intégrant divers menus et boutons qui facilitent la prise en main de jurassicWar par les joueurs.

4.4.1 Utilisation du module SDL

En premier lieu, la sélection du module SDL pour le projet se justifie par sa portabilité. Contrairement à d'autres librairies graphiques, SDL fonctionne à la fois sur Linux, Windows et macOS et est adapté au langage C. Il est principalement employé pour afficher le terrain, les dinosaures, les armes ainsi que divers menus explicités par la suite. Toutefois, il gère également les différentes actions du joueur tel qu'appuyer une touche du clavier pour choisir une arme et la trajectoire du tir dans les fonctions *choixHauteurLancer* et *viserArcher*, ou encore cliquer sur les boutons du menu principal dans la fonction *detecterEvenementsMenuPrincipal*. Sans oublier que les touches du clavier se veulent indispensables aux déplacements des dinosaures. Cependant, ce module représente également une contrainte puisqu'il requiert de gérer correctement la mémoire entre les différentes allocations et libérations que ce soit pour les fenêtres mais aussi pour les textures des dinosaures et des objets. En outre, obtenir un rendu propre lors du lancer de la bombe ou de l'envoi de flèches et des balles pendant les tirs exige des ajustements, notamment en ce qui concerne le délai d'affichage des images. D'un côté, un délai trop long engendre un visuel flou et peu esthétique pour l'utilisateur et d'un autre côté, un délai trop court consomme beaucoup de ressources. Par conséquent, cet intervalle de temps est directement lié à la vitesse des projectiles et donc aux calculs mathématiques liés à ceux-ci. Enfin, l'interface graphique de jurassicWar cumule une multitude d'images et de textes lorsque tous les élements du jeu c'est à dire l'inventaire et le terrain sont réunis. Ceci impose un ordre d'affichage strict pour assurer la fiabilité de l'affichage. C'est un enjeu rencontré par les différentes versions des fonctions *afficherJeu*.

4.4.2 Création d'un menu principal

En second lieu, un menu principal a été crée dans jurassicWar afin de fournir une interface intéractive pour le lancement d'une partie et l'accès aux principales informations du jeu.

4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur

5 Résultats et Conclusion

6 Bibliographie

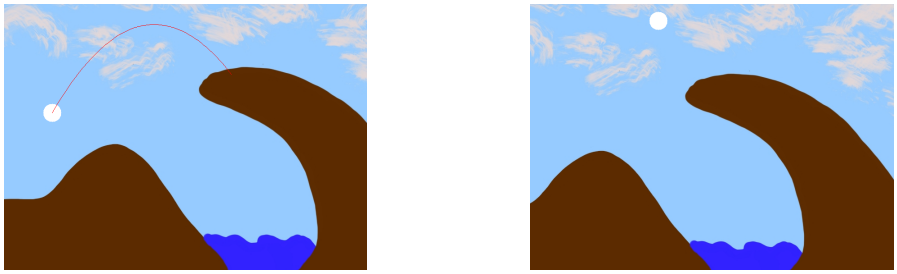
7 Annexes

7.1 Jeux de tests

7.2 Traces de débogage

7.2.1 Correction du système de collisions de la bombe

Lors de la création des fonctions chargées de détecter un contact entre la bombe et le terrain, la mer ainsi que les extrémités de la fenêtre, un bug silencieux a été rencontré, necessitant l'usage de l'outil gdb pour identifier le dysfonctionnement. La situation est la suivante :



(a) Trajectoire prévue pour la bombe (b) Arrêt anormal de la bombe

FIGURE 3 – Illustration du parcours incorrect effectué par la bombe

La bombe interrompt sa trajectoire au milieu du ciel avant d'avoir heurté la terre. Il s'agit par conséquent d'identifier une erreur dans une des fonctions liées à la bombe, en particulier dans la fonction *collisionTerrainBombe*. Pour cela, les fichiers impliqués dans l'affichage et le système de collisions sont compilés avec l'option -g :

```
CC = gcc

FLAGS_COMPILATION = -Wall -g `sdl2-config --cflags`
FLAGS_EDITION = `sdl2-config --libs` -lSDL2_image -lSDL2_ttf

PROG = ./bombe

OBJETS = \
    testFonctionsRebonds.o \
    fonctionsRebonds.o \
    fonctionsVerification.o \
    chargerMatrice.o

all: $(PROG)

$(PROG): $(OBJETS)
    $(CC) $(OBJETS) $(FLAGS_EDITION) -o $(PROG)

%.o: %.c
    $(CC) $(FLAGS_COMPILATION) -c $< -o $@
```

FIGURE 4 – Extrait du makefile utilisé pour le débogage des collisions

Les lignes qui suivent constituent la séquence des instructions de débogage mise en oeuvre pour corriger le système de collision. Cette séquence de commande nécessite de provoquer artificiellement l'évènement clavier déclenchant le lancer de la bombe afin d'analyser le contenu des variables lorsque l'arme est en mouvement puis lors de la collision. Dans le jeu, c'est la touche b du clavier, représentée par la macro SDL_SCANCODE_B dans le module SDL qui amorce le tir de la bombe. Cette macro possédant pour valeur cinq, l'élément d'indice cinq du tableau *etatClavier* qui consigne les touches actionnées par l'utilisateur est par conséquent positionné à un au cours de l'exécution du programme sous gdb.

Séquence des instructions de débogage :

```
gdb bombe
# Mise en place des points d'arrêt
# Initialiser l'affichage des variables
(gdb) break testFonctionsRebonds.c :67
# Simuler l'appui sur la touche b du clavier
(gdb) break testFonctionsRebonds.c :92
# Vérifier le contenu des variables au départ du lancement de la bombe
(gdb) break testFonctionsRebonds.c :98
# Analyser les valeurs des variables à la suite d'une collision avec le terrain
(gdb) break testFonctionsRebonds.c: 117
# Provoquer un arrêt en cas de collision avec le terrain
(gdb) break fonctionsRebonds.c:60 \
if ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE))
# Lancement de l'exécution du programme
(gdb) run
(gdb) display bombeLancee
bombeLancee = 0
(gdb) display bombe.coor
bombe.coor = {x = 325, y = 350}
(gdb) continue
bombeLancee = 0
bombe.coor = {x = 325, y = 350}
(gdb) set etatClavier[5] = 1
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 325, y = 350}
(gdb) continue
Thread 1 "bombe" hit Breakpoint 5, \
```

```
rebondTerrainBombe (matriceTerrain=0x7fffffc85290, bombe=0x7fffffc85234) \
at fonctionsRebonds.c:60 \
return ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE));
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 592, y = 131}
(gdb) next
bombeLancee = 0
bombe.coor = {x = 592, y = 131}
(gdb) quit
```

À l'issue de la séquence de commandes, il s'avère que l'erreur réside dans une inversion de coordonnées dans la fonction *collisionTerrainBombe*, comme l'illustre l'arrêt au point d'arrêt numéro cinq. Cette inversion s'explique par une différence de convention entre la matrice et l'affichage dans la fenêtre avec le module SDL. Tandis que la largeur correspond à l'axe des ordonnées pour la matrice, il s'agit de l'axe des abscisses pour l'affichage et inversement pour la hauteur. La résolution du comportement incohérent de la bombe nécessite donc uniquement d'échanger les coordonnées x et y dans les tests effectués par la fonction.